

AUTOMATIZACIÓN QA · MAGIIS

Automatización móvil con Appium

Visión general del desarrollo: qué cubrimos en las apps PAX y Driver, cómo se conecta con el resto de la automatización del proyecto, y cómo sumar cobertura.

Emanuel Restrepo · QA Automation, MAGIIS

25 de agosto de 2026 · Repo `magiis-testing` · Ambiente de referencia UAT

Inventario verificado contra el código, no contra documentación previa

1 · Qué cubre esta capa y qué no	alcance
2 · Inventario completo de cobertura	por feature, con estado real
3 · Cómo se conecta con el resto del proyecto	arquitectura
4 · Los dos modelos de trazabilidad	TM-* y MG-*
5 · Taxonomía de suites	dónde encaja Appium
6 · Nada de esto corre en CI	y por qué
7 · Cómo correr y cómo leer el resultado	operación
8 · Cómo sumar cobertura a un ticket	para QA automation
9 · Trampas conocidas del repo	lo que confunde

1 · Qué cubre esta capa y qué no

Appium automatiza las **dos apps móviles nativas** de MAGIIS: **App PAX** (pasajero) y **App Driver** (conductor). Los portales web — carrier, contractor, owner— no pasan por acá: van con Playwright puro contra el navegador.

37

CASOS XRAY DISTINTOS TRAZADOS

18

SPECS AUTOMATIZADAS

2

APPS CUBIERTAS

Es Android exclusivamente

Todo corre sobre el driver `uiautomator2` contra un teléfono Android físico conectado por USB.

No hay automatización de iOS

— ni XCUITest, ni simulador. Cuando hizo falta evidencia de iOS (campana MG-116), se resolvió capturando el payload a mano en el dispositivo y analizándolo con un script aparte (`scripts/mg116-analizar-payload-ios.mjs`). Es un workaround consciente, no cobertura automatizada.

Dos tipos de archivo que se parecen y no son lo mismo

Esta distinción es la que más confunde al llegar al repo, y decide si algo cuenta como cobertura:

Tipo	Cómo se reconoce	¿Es cobertura?
Spec	Importa <code>test / expect</code> de <code>@playwright/test</code> . La colecta un <code>playwright.*.config.ts</code> . Produce reporte Allure/Xray.	Sí
Script de diagnóstico	Termina en <code>run().catch(...)</code> . Se invoca a mano con <code>node --loader ts-node/esm</code> . Ningún config lo colecta.	No

Hay **106 scripts de diagnóstico** bajo `tests/mobile/appium/scripts/`. Son instrumentos de investigación — sirven para descubrir un selector, medir un comportamiento, reproducir un defecto. Varios *citan* keys de Xray en comentarios, y eso hace fácil contarlos por error como cobertura. **No lo son**: no emiten veredicto por el runner y no llegan a ningún reporte.

El par `prod-smoke-address-native.ts` + `prod-smoke-to-allure.mjs` sí produce evidencia trazada a Xray, por un camino lateral: el primero mide leyendo el árbol de accesibilidad de Android (en producción la WebView no es inspeccionable), el segundo convierte esa salida a formato Allure con enlaces `tms`. Es evidencia real, pero no es una spec.

2 · Inventario completo de cobertura

Verificado ejecutando el propio Playwright (`--list`) y leyendo las anotaciones del código, no la documentación previa. La columna de estado es lo importante: **una spec cuyos tests están en `test.fixme()` no aporta cobertura**, aunque exista y esté bien escrita.

2.1 · Autocomplete de direcciones — la campaña más reciente

Spec	App	Casos Xray	Estado
passenger/specs/pax-address-behaviors.spec.ts	PAX	TM-674, 675, 678, 679, 680, 681, 686, 687, 689, 697, 733, 734	activa
passenger/specs/pax-address-surfaces.spec.ts	PAX	TM-727, 729, 730, 731	activa
driver/specs/driver-address-surfaces.spec.ts	Driver	TM-650, 651, 654, 655, 656, 657, 662, 663, 664, 665	activa

26 casos Xray, todos con anotación `tms` real. Es la cobertura mejor instrumentada de la capa: mide comportamiento del cliente (agrupamiento de pulsaciones, piso de caracteres, reutilización y rotación del token de sesión) capturando el tráfico dentro de la WebView.

Último resultado medido — 24 de agosto de 2026, UAT

La superficie principal de PAX (Home · Origen) midió sus **9 conductas en verde**

. Un solo rojo en toda la corrida:

TM-731

, un defecto de producto confirmado — recargar la WebView en `travel-info` deja la app en la página de error de Chrome.

2.2 · Pagos y wallet — cobertura previa, sobre el gateway

Vive en otro árbol (`tests/features/gateway-pg/specs/`) y ningún documento la mencionaba hasta ahora, aunque depende del mismo motor Appium.

Spec	Qué cubre	Casos Xray	Estado
apppax-wallet-delete	Borrar una tarjeta 3DS vinculada de la wallet	MG-174, MG-495	activa
apppax-wallet-management	Gestión de la wallet del pasajero	MG-295, MG-302	activa
viaje-calle/driver-viaje-calle-3ds	Viaje de calle → cobro a bordo con 3DS → vuelta al home	MG-161	activa
_parametrized/apppax-wallet.parametrized	Alta de tarjeta multi-pasarela, parametrizado	MG-148, MG-288, MG-293, MG-295	activa
cargo-a-bordo/apppax-cargo-asignado-success cargo-a-bordo/apppax-cargo-asignado-3ds	Cobro a bordo asignado, con y sin 3DS	MG-158, MG-161	parcial
apppax-personal-3ds · -no3ds apppax-business-3ds · -no3ds apppax-business-hold-3ds · -no3ds	Alta de tarjeta y hold, wallet personal y de colaborador	MG-148, 152, 153, 158, 161	mayormente fixme
carrier-driver-happy-path-template	Happy path carrier web → app Driver, data-driven	ninguna	1 de 3 escenarios

Seis specs de alta de tarjeta están escritas pero no corren

Sus tests están marcados `test.fixme()` esperando validación manual en dispositivo. Existen, compilan y tienen trazabilidad declarada — pero

no producen evidencia

. Contarlas como cobertura sería el error más fácil de cometer al leer este repo, y es la razón por la que este inventario lleva columna de estado.

2.3 · Flujos híbridos — web y móvil en una sola prueba

Spec	Qué cubre	Estado
tests/e2e/gateway/flow1-carrier-driver	Alta desde carrier web → el driver completa en la app	activa, sin trazabilidad
tests/e2e/gateway/flow3-contractor-driver	Igual, originado por contractor	activa, sin trazabilidad
tests/e2e/gateway/flow2-passenger-driver	Alta desde el pasajero	100 % fixme
tests/e2e/flow1-carrier-driver.e2e.spec.ts	Versión legacy de flow1, aún presente	legacy

Ninguno de los cuatro traza casos de Xray. Sus títulos usan identificadores propios (FLOW1-TC01) que no son keys de Jira.

3 · Cómo se conecta con el resto del proyecto

La clave para entender esta capa: **Appium comparte el runner con todo el resto, pero no el driver.**

```
Playwright = el runner (test(), expect(), hooks, anotaciones, reporters, tags, retries)
|
+-- specs web  ———> driver = Playwright (page, browser, contexts)
|
+-- specs móvil —> driver = WebdriverIO (remote() -> Appium -> uiautomator2 -> teléfono)
```

Una spec de Appium importa `test` y `expect` de Playwright igual que cualquier otra —por eso funcionan las anotaciones, Allure y el reporter de Xray sin nada especial— pero abre la sesión con `remote()` de WebdriverIO y **nunca usa las fixtures `page` ni `browser`**. Los configs de dispositivo declaran `storageState` vacío justamente para no arrastrar la autenticación de los portales web.

Tres arquitecturas conviven dentro de la capa móvil

Patrón	Dónde	Cómo funciona
La spec maneja su sesión	autocomplete (MG-116/117)	Abre la sesión, corre una batería de mediciones, cada test lee su resultado. Control total, sesión pagada una vez.
Screen objects + harness	gateway <code>e2e-mobile</code>	La spec orquesta objetos de pantalla y recorridos reutilizables (<code>harness/</code>). Más parecido al patrón POM del resto del repo.
Proceso hijo	<code>tests/e2e/gateway/flow1, flow3</code>	La fase web corre en el proceso de Playwright y lanza la fase móvil como proceso aparte , parseando su salida estándar. El estado del viaje se pasa por un archivo de contexto.

Y una capa que no comparte: la arquitectura KATA

El repo tiene una arquitectura por capas llamada KATA (`TestContext` -> `ApiBase/UiBase` -> `componentes` -> `fixtures`) que usan las suites de API y las pantallas web nuevas. **La capa móvil no la extiende:** `AppiumSessionBase` es una clase independiente que envuelve WebdriverIO y no comparte jerarquía con esa estructura. Tiene su propia organización paralela por directorios (`config/` , `base/` , `Screens`, `harness/` , `helpers/`).

4 · Los dos modelos de trazabilidad

Dentro de la misma capa Appium conviven **dos convenciones distintas** para vincular una prueba con su caso en Jira, y ninguna documentación del repo lo explicaba. Confundirlas lleva a leer mal un reporte.

	TM-* · autocomplete	MG-* · gateway
Dónde viven los casos	Proyecto Xray separado (TM)	El mismo proyecto de producto (MG)
Granularidad	Un caso por conducta medida	Una key por <i>área</i> funcional
Dónde se declara	En cada <code>test()</code>	A nivel <code>describe</code>
Cómo se lee el resultado	Un veredicto por conducta	Todos los tests del área colapsan a una fila

El detalle que hay que entender antes de leer un reporte de gateway

El reporter deduplica por key y **gana el peor estado**

. Con la convención por área, veinte tests que comparten una key colapsan en una sola fila, y esa fila refleja el *peor* de los veinte. Un rojo entre veinte pinta toda el área de rojo. No es un bug del reporter: es la consecuencia de trazar por área en vez de por caso — y explica por qué el autocomplete eligió la convención granular.

Cómo una anotación llega a ser un resultado en Xray

```
// 1. En el test - la forma explícita
test('...', { annotation: [{ type: 'tms', description: 'TM-678' }] }, async () => { ... });

// 2. O vía decorador, en un componente KATA (empuja la anotación en runtime)
@atc('MG-152', { severity: 'critical', description: '...' })
```

El reporter junta las anotaciones estáticas y las de runtime, recoge **todas** las keys distintas (una prueba puede acreditar varios casos) y las escribe al JSON de importación. Si una prueba no tiene ninguna anotación, cae a un respaldo que busca algo con forma de key *en el título* — y ese respaldo captura **una sola** key y no genera el enlace a Jira en Allure. Por eso la anotación explícita no es opcional.

5 · Taxonomía de suites

Config	Qué colecta	Tipo
playwright.config.ts	Todo el árbol de tests; hace login web de carrier y contractor al arrancar	Web UI general
playwright.gateway-pg.config.ts	Suite de pagos, dividida en proyectos: web, API, unit, visual y e2e-mobile	Mixta — acá viven las specs móviles de gateway
playwright.mg116-device.config.ts	Solo el autocomplete de PAX. Sin login web, un worker, sin reintentos	Dispositivo
playwright.mg117-device.config.ts	Solo el autocomplete de Driver. Igual criterio	Dispositivo

Nunca pasar `--reporter=` por línea de comandos

Ese flag reemplaza la lista completa de reporters del config, incluido el de Xray. La corrida se ve perfecta y el JSON de trazabilidad nunca se escribe. Para ver salida tipo lista conservando el reporter, usar las variables (`XRAY=1` , `ALLURE=1`).

6 · Nada de esto corre en CI

Conviene decirlo sin rodeos, porque es lo primero que alguien nuevo asume mal: **ninguna prueba de Appium corre en los pipelines**. Toda la capa es de ejecución local, contra un teléfono conectado por USB.

Es una decisión, no un olvido: el pipeline de gateway excluye explícitamente las specs móviles porque sin un emulador disponible la corrida rompe. Hay una propuesta de CI para Android en [docs/mobile-ci/](#), pero es una propuesta, no un pipeline implementado.

Qué implica en la práctica: esta cobertura no protege contra regresiones de forma automática. Protege cuando alguien la corre. Planificarla como gate de release exige agendar la corrida, no asumirla.

7 · Cómo correr y cómo leer el resultado

1 Verificar el stack antes de correr

```
adb devices          # el teléfono tiene que aparecer
curl localhost:4723/status  # Appium responde
```

2 Correr la suite

```
# autocomplete App PAX
pnpm test:uat:mg116:all

# autocomplete App Driver
pnpm test:uat:mg117:all

# pagos / wallet / cargo a bordo (proyecto e2e-mobile)
npx playwright test -c playwright.gateway-pg.config.ts --project=e2e-mobile
```

3 Leer el resultado — sin confiar en el banner

El resumen final no es criterio de éxito

Un cartel de "ALL TESTS PASSED" puede estar contando cuatro pruebas que pasaron y veinticinco que se saltaron — pasó en esta campaña. Lo que hay que leer es Automatización móvil Appium — MAGI3 **cuántos casos tienen veredicto medido**

: un skip con motivo escrito es honesto, pero no es cobertura. El JSON de resultados es la fuente, el banner es decoración.

Herramienta	Para qué
Consola	Veredicto inmediato con el mensaje de cada medición
Allure (<code>ALLURE=1</code>)	Reporte navegable con capturas y enlace al ticket
Xray (<code>XRAY=1</code>)	Genera el JSON para importar y actualizar los casos en Jira

El archivo de resultados se sobrescribe en cada corrida. Si la evidencia importa, fijar `XRAY_OUTPUT_FILE` con un nombre propio antes de correr.

8 · Cómo sumar cobertura a un ticket

1 Primero: decidir la capa

La pregunta que decide

¿El aserto sigue teniendo sentido si el propio test construye la petición?

Si

no

—porque lo que se prueba es qué manda la app, cuándo la manda, qué guarda entre pantallas— es conducta del cliente y va al dispositivo.

Si

sí

—porque lo que se prueba es qué contesta el backend— es un hecho del response: va a la capa de API, corre en segundos y no necesita teléfono.

Nota honesta: este criterio no estaba escrito en ningún lado del repo hasta este documento. Se formuló durante la campaña de autocomplete, cuando se descubrió que varios casos atribuidos a la capa de API en realidad verificaban pantalla. Vale como guía, no como política heredada.

2 Buscar antes de escribir

La mayoría de las conductas ya están implementadas y solo hace falta invocarlas sobre una superficie nueva. Agregar una superficie cuesta una clase; escribir la conducta de nuevo cuesta un duplicado que va a divergir.

3 Trazar con anotación explícita

```
test(
  'descripción de la conducta',
  { annotation: [{ type: 'tms', description: 'TM-XXX' }] },
  async () => { /* ... */ }
);
```

Nunca confiar en el respaldo por título: captura una sola key y no genera el enlace en Allure.

4 Respetar la disciplina de veredicto

Situación	Veredicto correcto
Se midió y cumple	PASS
Se midió y no cumple	FAIL
No se pudo medir (pantalla inalcanzable, sin datos)	SIN_DATOS / NO_EJERCIDO — nunca FAIL
La medición no puede distinguir producto de harness	NO_EJERCIDO , con el motivo escrito

Nunca un verde sobre una ausencia

"Cero requests" puede significar "el producto agrupa bien" o "el campo dejó de aceptar texto" — son indistinguibles sin una prueba de vida. Ya pasó en este repo: dos conductas se publicaron en verde sobre un campo inerte. Si el veredicto se apoya en que algo *no* ocurrió, hay que probar primero que el campo seguía vivo.

5 Verificar antes de reportar

```
npx tsc --noEmit # compila
npx playwright test --config=... --list # colecta lo esperado
```

6 Importar a Xray solo si la corrida midió

Automatización móvil Appium — MAGIIS
Si la mayoría quedó en skip, el JSON no se importa: escribiría estados que nadie verificó.

9 · Trampas conocidas del repo

Cosas que parecen una cosa y son otra. Todas verificadas leyendo el código.

Qué	Realidad
<code>tests/mobile/appium/recorded/*.recorded.ts</code>	No es Appium. Son volcados crudos del codegen de Playwright sobre un portal <i>web</i> . Ningún config los colecta. Están mal ubicados bajo el árbol móvil.
<code>tests/mobile/appium/gateway-pg/*Draft.ts</code>	Código muerto. Nadie los importa; uno de sus métodos es literalmente vacío. No están conectados a nada.
<code>passenger/specs/pax-new-trip-blocked.spec.ts</code>	Spec real y bien escrita, pero huérfana : ningún config la colecta y no tiene anotaciones. Su propio encabezado se declara borrador pendiente de validación.
Scripts que citan keys TM-*	Citarlas en un comentario no las convierte en cobertura. Son instrumentos de investigación.
Un archivo web dentro de <code>e2e-mobile</code>	El proyecto selecciona por etiqueta, así que una spec de carrier web etiquetada igual también entra. Explica por qué el conteo del proyecto es mayor que el de archivos móviles.

Dónde seguir

La referencia técnica viva del motor —instalación paso a paso, anatomía de directorios, verificación del stack, glosario— está en `tests/mobile/appium/README.md`, junto al código. Este documento da la visión general; ese otro es el manual de operación.